

PostgreSQL offers built-in support for the following forms of partitioning:

Range Partitioning

The table is partitioned into “ranges” defined by a key column or set of columns, with no overlap between the ranges of values assigned to different partitions. For example, one might partition by date ranges, or by ranges of identifiers for particular business objects. Each range's bounds are understood as being inclusive at the lower end and exclusive at the upper end. For example, if one partition's range is from 1 to 10, and the next one's range is from 10 to 20, then value 10 belongs to the second partition not the first.

List Partitioning

The table is partitioned by explicitly listing which key value(s) appear in each partition.

Hash Partitioning

The table is partitioned by specifying a modulus and a remainder for each partition. Each partition will hold the rows for which the hash value of the partition key divided by the specified modulus will produce the specified remainder.

Alternative methods such as inheritance and UNION ALL views can be used instead. Such methods offer flexibility but do not have some of the performance benefits of built-in **declarative partitioning**.

PostgreSQL allows you to declare that a table is divided into partitions. The table that is divided is referred to as a *partitioned table*. The declaration includes the *partitioning method* as described above, plus a list of columns or expressions to be used as the *partition key*.

Partitions can also be **foreign tables**, although considerable care is needed because it is then the user's responsibility that the contents of the foreign table satisfy the partitioning rule.

Example

Suppose we are constructing a database for a large ice cream company. The company measures peak temperatures every day as well as ice cream sales in each region.

```
CREATE TABLE measurement (  
  city_id      int not null,  
  logdate      date not null,  
  peaktemp     int,  
  unitsales    int  
);
```

To reduce the amount of old data that needs to be stored, we decide to keep only the most recent 3 years worth of data.

At the beginning of each month we will remove the oldest month's data.

In this situation we can use partitioning to help us meet all of our different requirements for the measurements table.

To use declarative partitioning in this case, use the following steps:

1. Create the measurement table as a partitioned table by specifying the PARTITION BY clause, which includes the partitioning method (RANGE in this case) and the list of column(s) to use as the partition key.

```
CREATE TABLE measurement (  
    city_id      int not null,  
    logdate      date not null,  
    peaktemp     int,  
    unitsales    int  
) PARTITION BY RANGE (logdate);
```

2. Create partitions. Each partition's definition must specify bounds that correspond to the partitioning method and partition key of the parent. Note that specifying bounds such that the new partition's values would overlap with those in one or more existing partitions will cause an error.

Partitions thus created are in every way normal PostgreSQL tables.

For our example, each partition should hold one month's worth of data, to match the requirement of deleting one month's data at a time. So the commands might look like:

```
CREATE TABLE measurement_y2006m02 PARTITION OF measurement  
FOR VALUES FROM ('2006-02-01') TO ('2006-03-01');
```

```
CREATE TABLE measurement_y2006m03 PARTITION OF measurement  
FOR VALUES FROM ('2006-03-01') TO ('2006-04-01');
```

...

```
CREATE TABLE measurement_y2007m11 PARTITION OF measurement  
FOR VALUES FROM ('2007-11-01') TO ('2007-12-01');
```

```
CREATE TABLE measurement_y2007m12 PARTITION OF measurement  
FOR VALUES FROM ('2007-12-01') TO ('2008-01-01')  
TABLESPACE fasttablespace;
```

```
CREATE TABLE measurement_y2008m01 PARTITION OF measurement  
FOR VALUES FROM ('2008-01-01') TO ('2008-02-01')  
WITH (parallel_workers = 4)  
TABLESPACE fasttablespace;
```

```
CREATE TABLE measurement_y2006m02 PARTITION OF measurement  
FOR VALUES FROM ('2006-02-01') TO ('2006-03-01')
```

PARTITION BY RANGE (peaktemp);

After creating partitions of measurement_y2006m02, any data inserted into measurement that is mapped to measurement_y2006m02 (or data that is directly inserted into measurement_y2006m02, which is allowed provided its partition constraint is satisfied) will be further redirected to one of its partitions based on the peaktemp column.

The partition key specified may overlap with the parent's partition key, although care should be taken when specifying the bounds of a sub-partition such that the set of data it accepts constitutes a subset of what the partition's own bounds allow; the system does not try to check whether that's really the case.

Inserting data into the parent table that does not map to one of the existing partitions will cause an error; an appropriate partition must be added manually.

3. Create an index on the key column(s), as well as any other indexes you might want, on the partitioned table. (The key index is not strictly necessary, but in most scenarios it is helpful.) This automatically creates a matching index on each partition, and any partitions you create or attach later will also have such an index. An index or unique constraint declared on a partitioned table is “virtual” in the same way that the partitioned table is: the actual data is in child indexes on the individual partition tables.

CREATE INDEX ON measurement (logdate);

4. Ensure that the [enable_partition_pruning](#) configuration parameter is not disabled in postgresql.conf. If it is, queries will not be optimized as desired.

In the above example we would be creating a new partition each month, so it might be wise to write a script that generates the required DDL automatically.

Partition Maintenance

The simplest option for removing old data is to drop the partition that is no longer necessary:

DROP TABLE measurement_y2006m02;

Another option that is often preferable is to remove the partition from the partitioned table but retain access to it as a table in its own right. This has two forms:

**ALTER TABLE measurement DETACH PARTITION measurement_y2006m02;
ALTER TABLE measurement DETACH PARTITION measurement_y2006m02
CONCURRENTLY;**

Similarly we can add a new partition to handle new data.

We can create an empty partition in the partitioned table just as the original partitions were created above:

```
CREATE TABLE measurement_y2008m02 PARTITION OF measurement
FOR VALUES FROM ('2008-02-01') TO ('2008-03-01')
TABLESPACE fasttablespace;
```

```
CREATE TABLE measurement_y2008m02
(LIKE measurement INCLUDING DEFAULTS INCLUDING CONSTRAINTS)
TABLESPACE fasttablespace;
```

```
ALTER TABLE measurement_y2008m02 ADD CONSTRAINT y2008m02
CHECK ( logdate >= DATE '2008-02-01' AND logdate < DATE '2008-03-01' );
```

Partitioning Using Inheritance

While the built-in declarative partitioning is suitable for most common use cases, there are some circumstances where a more flexible approach may be useful. Partitioning can be implemented using table inheritance, which allows for several features not supported by declarative partitioning, such as:

- For declarative partitioning, partitions must have exactly the same set of columns as the partitioned table, whereas with table inheritance, child tables may have extra columns not present in the parent.
- Table inheritance allows for multiple inheritance.
- Declarative partitioning only supports range, list and hash partitioning, whereas table inheritance allows data to be divided in a manner of the user's choosing. (Note, however, that if constraint exclusion is unable to prune child tables effectively, query performance might be poor.)

Example

This example builds a partitioning structure equivalent to the declarative partitioning example above. Use the following steps:

1. Create the “root” table, from which all of the “child” tables will inherit. This table will contain no data. Do not define any check constraints on this table, unless you intend them to be applied equally to all child tables. There is no point in defining any indexes or unique constraints on it, either. For our example, the root table is the measurement table as originally defined:

```
CREATE TABLE measurement (
  city_id      int not null,
  logdate      date not null,
  peaktemp     int,
  unitsales    int
);
```

2. Create several “child” tables that each inherit from the root table. Normally, these tables will not add any columns to the set inherited from the root. Just as with declarative partitioning, these tables are in every way normal PostgreSQL tables (or foreign tables).
3. CREATE TABLE measurement_y2006m02 () INHERITS (measurement);

4. CREATE TABLE measurement_y2006m03 () INHERITS (measurement);
5. ...
6. CREATE TABLE measurement_y2007m11 () INHERITS (measurement);
7. CREATE TABLE measurement_y2007m12 () INHERITS (measurement);
8. CREATE TABLE measurement_y2008m01 () INHERITS (measurement);
9. Add non-overlapping table constraints to the child tables to define the allowed key values in each.

Creating object types v16

You can use the `CREATE TYPE` command to create an object type specification and the `CREATE TYPE BODY` command to create an object type body. The examples that follow use the `CREATE TYPE` and `CREATE TYPE BODY` commands.

The first example creates the `addr_object_type` object type that contains only attributes and no methods:

```
CREATE OR REPLACE TYPE addr_object_type AS OBJECT
(
  street    VARCHAR2(30),
  city      VARCHAR2(20),
  state     CHAR(2),
  zip       NUMBER(5)
);
```